LAB 3 | Adaptive optimizers for ANN and MLP

MNIST HANDWRITTEN DIGITS

Train a shallow ANN and a deeper MLP using the same public image dataset, swapping only the optimizer constructor while keeping the architecture and fit parameters fixed.

Learning objectives

- Tell ANN with one hidden layer from a deeper MLP (both are feedforward ANNs).
- Make five optimizer changes with valid tf.keras class names.
- Compare validation accuracy curves, held-out accuracy and training time.

Reproducible experiment settings

- Dataset MNIST | training 3000 | validation 600 | test 1200.
- Architecture 1: Flatten → Dense(64) → Dense(10); architecture 2: Flatten → Dense(128) → Dense(64) → Dense(10).
- Fixed EPOCHS=4 and BATCH_SIZE=64; compare SGD, Adam, RMSprop, Adadelta and Adagrad.
- Set initial model weights equal across optimizer runs.

Run instructions

1. In Google Colab, create a notebook, paste the complete Python solution and run it.
2. For local Python, install TensorFlow, NumPy and Matplotlib: pip install tensorflow numpy matplotlib
3. The public dataset downloads on the first execution. Actual performance numbers are created only when the code is run.
4. The Python (.py) source file is provided separately for direct execution and easier copying.

Complete worked solution | line-commented Python code

Instructional comments accompany each executable statement. The standalone .py file contains exactly this code; output numbers and charts are generated by running it.

```python
# LAB 1: Compare SGD, Adam, RMSprop, Adadelta, and Adagrad on ANN and MLP.
# Run in Google Colab or locally after: pip install tensorflow numpy matplotlib
# Dataset: MNIST handwriting images; downloaded automatically on first use.

import os  # Access an environment setting that reduces TensorFlow log noise.
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"  # Show fewer TensorFlow informational messages.
import csv  # Export an easy-to-read results table without extra packages.
import time  # Measure how long each training experiment takes.
from pathlib import Path  # Create a folder for result files on any operating system.
import numpy as np  # Select the same images for every experiment.
import matplotlib.pyplot as plt  # Create accuracy comparison graphs.
import tensorflow as tf  # Build, train, and test neural networks.

SEED = 42  # Use a repeatable random seed for classroom comparisons.
TRAIN_N = 3000  # Limit training images so the lab runs on a normal laptop.
VAL_N = 600  # Keep separate validation images for model selection.
TEST_N = 1200  # Keep a separate test subset for the final evaluation.
EPOCHS = 4  # Set the same training epochs for every optimizer.
BATCH_SIZE = 64  # Keep mini-batch size fixed while comparing optimizers.
OUT = Path("lab1_optimizer_results")  # Name the results folder.
OUT.mkdir(parents=True, exist_ok=True)  # Create the folder if needed.
tf.keras.utils.set_random_seed(SEED)  # Set Python, NumPy, and TensorFlow seeds.

(x_all, y_all), (x_test_all, y_test_all) = tf.keras.datasets.mnist.load_data()  # Load public MNIST.
rng = np.random.default_rng(SEED)  # Create a deterministic NumPy random generator.
train_order = rng.permutation(len(x_all))  # Shuffle original training indices one time.
test_order = rng.permutation(len(x_test_all))  # Shuffle separate test indices one time.
selected = train_order[:TRAIN_N + VAL_N]  # Select nonoverlapping train and validation data.
x_train = x_all[selected[:TRAIN_N]].astype("float32") / 255.0  # Scale training pixels to 0..1.
y_train = y_all[selected[:TRAIN_N]]  # Select training labels using matching indices.
x_val = x_all[selected[TRAIN_N:]].astype("float32") / 255.0  # Scale validation pixels.
y_val = y_all[selected[TRAIN_N:]]  # Select validation labels.
x_test = x_test_all[test_order[:TEST_N]].astype("float32") / 255.0  # Scale held-out test pixels.
y_test = y_test_all[test_order[:TEST_N]]  # Select matching test labels.

# A shallow ANN is also technically an MLP; here ANN means one hidden layer.
def build_ann():  # Define the smaller one-hidden-layer network.
    return tf.keras.Sequential([  # Construct a Keras network with ordered layers.
        tf.keras.layers.Input(shape=(28, 28)),  # Receive one 28-by-28 image.
        tf.keras.layers.Flatten(),  # Convert the image into 784 numeric features.
        tf.keras.layers.Dense(64, activation="relu"),  # Learn 64 hidden features.
        tf.keras.layers.Dense(10, activation="softmax"),  # Predict 10 digit classes.
    ])  # End the ANN model definition.

# In this lab, MLP means a deeper multilayer perceptron with two hidden layers.
def build_mlp():  # Define a two-hidden-layer multilayer perceptron.
    return tf.keras.Sequential([  # Construct the second Keras network.
        tf.keras.layers.Input(shape=(28, 28)),  # Receive a grayscale image.
        tf.keras.layers.Flatten(),  # Flatten 28-by-28 pixels into 784 features.
        tf.keras.layers.Dense(128, activation="relu"),  # Learn 128 first-layer features.
        tf.keras.layers.Dense(64, activation="relu"),  # Combine them in 64 deeper units.
        tf.keras.layers.Dense(10, activation="softmax"),  # Produce class probabilities.
```

```
    ])  # End the MLP model definition.

# These are illustrative starting learning rates, not necessarily Keras defaults.
OPTIMIZERS = [  # Define the ONLY optimizer line that must change between experiments.
    ("SGD", lambda: tf.keras.optimizers.SGD(learning_rate=0.01)),  # Fixed-rate baseline.
    ("Adam", lambda: tf.keras.optimizers.Adam(learning_rate=0.001)),  # Adaptive moments.
    ("RMSprop", lambda: tf.keras.optimizers.RMSprop(learning_rate=0.001)),  # Running square average.
    ("Adadelta", lambda: tf.keras.optimizers.Adadelta(learning_rate=1.0)),  # Adaptive update scaling.
    ("Adagrad", lambda: tf.keras.optimizers.Adagrad(learning_rate=0.01)),  # Accumulated gradients.
]  # Finish the optimizer configurations.

results = []  # Collect a summary row for each model-and-optimizer combination.
for model_name, builder in [("ANN", build_ann), ("MLP", build_mlp)]:  # Test both architectures.
    tf.keras.utils.set_random_seed(SEED)  # Reproduce initial weights for this architecture.
    reference_model = builder()  # Create one reference model to record its initial weights.
    initial_weights = reference_model.get_weights()  # Store the same starting weights for all runs.
    del reference_model  # Remove the reference because it is not trained.
    histories = {}  # Save validation accuracy curves for this architecture.
    for opt_name, make_optimizer in OPTIMIZERS:  # Repeat for all five optimization algorithms.
        tf.keras.utils.set_random_seed(SEED)  # Reset random ordering and experiment seeds.
        model = builder()  # Make an independent new model for this optimizer.
        model.set_weights(initial_weights)  # Guarantee identical initial model weights.
        optimizer = make_optimizer()  # Change only this optimizer choice across runs.
        model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy", metrics=["accuracy"])  # Configure training.
        start = time.perf_counter()  # Start a wall-clock timer immediately before training.
        history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=EPOCHS, batch_size=BATCH_SIZE, shuffle=True, verbose=0)  # Train without verbose logs.
        seconds = time.perf_counter() - start  # Calculate elapsed training seconds.
        test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)  # Evaluate unseen test images.
        best_val = max(history.history["val_accuracy"])  # Identify the best validation accuracy.
        histories[opt_name] = history.history["val_accuracy"]  # Store the full accuracy trend.
        results.append({"model": model_name, "optimizer": opt_name, "learning_rate": float(optimizer.learning_rate.numpy()), "best_val_accuracy": round(best_val, 4), "test_accuracy":
round(test_acc, 4), "test_loss": round(test_loss, 4), "train_seconds": round(seconds, 2)})  # Save numeric outcomes.
        print(f"{model_name:3} | {opt_name:8} | val={best_val:.3f} | test={test_acc:.3f} | {seconds:.1f}s")  # Display real measured results.
    plt.figure(figsize=(8, 4.5))  # Start an individual accuracy figure for this architecture.
    for opt_name, val_curve in histories.items():  # Add one line for each optimizer.
        plt.plot(range(1, EPOCHS + 1), val_curve, marker="o", label=opt_name)  # Draw its true validation history.
    plt.title(f"{model_name}: validation accuracy by optimizer")  # Identify the experiment.
    plt.xlabel("Epoch")  # Label horizontal position as epochs.
    plt.ylabel("Validation accuracy")  # Label vertical position as validation accuracy.
    plt.xticks(range(1, EPOCHS + 1))  # Show whole-number epoch ticks.
    plt.grid(alpha=0.25)  # Add subtle guidelines for easier comparisons.
    plt.legend()  # Identify each optimizer curve.
    plt.tight_layout()  # Prevent figure labels from being cut off.
    plt.savefig(OUT / f"{model_name.lower()}_validation_accuracy.png", dpi=160)  # Save the result graph.
    plt.close()  # Release figure memory before the next model.

with (OUT / "optimizer_results.csv").open("w", newline="", encoding="utf-8") as file:  # Open a CSV output file.
    writer = csv.DictWriter(file, fieldnames=list(results[0].keys()))  # Use dictionary columns for the table.
    writer.writeheader()  # Write descriptive column names.
    writer.writerows(results)  # Write every actual measured model result.
print(f"Results and plots saved to: {OUT.resolve()}")  # Show where to find files.
```

Expected generated files

- lab1_optimizer_results/optimizer_results.csv
- lab1_optimizer_results/ann_validation_accuracy.png

- lab1_optimizer_results/mlp_validation_accuracy.png

How to interpret your results (sample answer framework)

Which optimizer is best?

Was the comparison controlled?

Why are initial rates different?

Interpret the ANN vs MLP contrast